

AGENT_ARCHITECTURE_PLAIN

Helios Agent Architecture — In Plain English

Plain-language companion to `docs/architecture/AGENT_ARCHITECTURE.md`. For the exact spec, read that / its PDF.

In one sentence

Helios is not one free-roaming AI — it's a fixed assembly line (a plain-Python state machine) where each station is a Claude agent with one job, a locked toolset, and a strict rule that nothing ships until the Critic tries to tear it apart.

Why this matters to you

This is where "AI growth analyst" stops being a vibe and becomes a reproducible machine. The key inversion: **the Python runner is the boss; the LLM is just a worker that fills in one station.** That choice is what makes runs auditable, unit-testable with recorded fixtures, and gradeable by an offline benchmark. It's also why "the agent went rogue and wrote its own SQL" can't happen — no agent holds a raw-SQL tool. If you understand this control-plane-vs-node split, everything else clicks.

The big ideas, simply

Deterministic control plane, model-driven nodes. A plain-Python finite state machine (the FSM) decides what happens next. At each state it invokes one agent with a fixed prompt and a fixed tool allow-list. The agent loops tool calls, then returns a **typed JSON envelope** (a `Finding`). The runner reads that envelope and picks the next state. The model never controls the flow.

The seven agents:

Agent	Model	Plain job
Orchestrator	Opus	Plans the run, sets scope + budget, drives the FSM, routes to the Critic
Monitor	Sonnet	Spots which metric/segment moved abnormally
Decompose	Sonnet	Splits each move into mix / rate / interaction
Diagnose	Opus	Best-first hypothesis search to find the root cause; prices it in dollars
Critic	Opus	Tries to <i>refute</i> every candidate finding before it ships
Prescribe	Sonnet	Turns survivors into properly-sized experiment cards
Narrator	Sonnet	Writes the executive brief; saves the diagnosis to memory

Why Opus for three roles: Opus goes to **Orchestrator, Diagnose, Critic** — the jobs with an open-ended hypothesis space and high cost of error. Sonnet runs the four bounded jobs (Monitor, Decompose, Prescribe, Narrator) where the next move is largely determined. This keeps token cost down without hurting accuracy where it counts.

Nothing ships unrefuted. The Critic's whole job is to *attack*: is this really a mix-shift, not a behavior change? Is the sample too small? Is it just known seasonality? Is it a data-quality glitch? Verdicts are **PASS / DOWNGRADE / DROP**. Only PASS findings reach the brief.

What you actually build / how it works

- **The FSM:** PLAN → MONITOR → DECOMPOSE → DIAGNOSE → CRITIC (per finding) → PRESCRIBE → NARRATE → END . A clean run (no anomaly) short-circuits straight to a "no material change" brief — and still records that it ran.
- **The tool-call wrapper** is where the invariants live. Every single tool call passes through it, and it: checks the tool is in the agent's allow-list, asserts `dry_run` ran before `run_query` (rule G3), enforces the running 5 GiB byte budget mid-run, retries transient errors (≤ 3 backoffs), and writes an `audit_log` row. This one wrapper is what proves "100% governed SQL."
- **The Finding envelope** is the spine — every hand-off between agents is a typed JSON `Finding`, never prose. It carries the metric, the dimension slice, the t0/t1 windows, the decomposition, the significance test, the dollar impact, the evidence, and the Critic's verdict.
- **Diagnose is a best-first tree search.** It always drills the slice with the largest **rate-effect** first (real behavior change), not the largest mix-effect (composition artifact / Simpson's paradox). It prunes branches that are insignificant, low-effect, or low-sample, and promotes

a leaf to a candidate finding only if it's significant, single-dimension, reconciled ($\leq 0.5\%$ drift), and dollar-material.

- **Revenue-at-risk at the leaf:** $(r_{\text{counterfactual}} - r_{\text{observed}}) \times \text{affected_sessions} \times \text{downstream_value}$ — e.g. $(\text{conv_t0} - \text{conv_t1}) \times \text{sessions_t1} \times \text{aov}$. Computed entirely from governed metrics, never in prose.
- **Context stays small.** Each node gets a compacted context (run plan, metric/dimension names only, upstream `Finding[]`, recalled priors). Raw query rows are *never* dumped into the LLM — they're reduced to aggregates first, so token cost stays flat as the tree widens.
- **Build order:** the framework first, then the M7 minimal loop (Monitor + Narrator + runner), then the full M9 loop (Decompose, Diagnose, Critic, Prescribe, Orchestrator, memory).

Easy things to get wrong

- **Thinking the LLM drives the loop.** It doesn't — the Python FSM does. The agent only returns an envelope.
- **Chasing mix-effects.** Diagnose drills the largest *rate* effect first; mix-effects are composition artifacts.
- **Shipping a finding the Critic didn't clear.** Worker output never ships directly; it must survive PASS.
- **Expecting byte-identical reruns.** SQL and stats are deterministic; LLM nodes at temperature 0 are *not* byte-deterministic — so the agent layer is graded **statistically** ($\geq 85\%$ root-cause accuracy), with the full audit log making every run reconstructable.
- **Shipping a partial brief on failure.** A hard failure ABORTs with an audit row and **no partial brief**.

Glossary — the exact words, demystified

- **FSM** — finite state machine; the plain-Python runner that sequences the stages.
- **Finding** — the typed JSON envelope agents pass to each other.
- **mix / rate / interaction** — the three parts a metric change splits into; $\Delta R = \text{mix} + \text{rate} + \text{interaction}$.
- **PASS / DOWNGRADE / DROP** — the Critic's three verdicts.
- **revenue-at-risk** — the dollar size of a finding, computed from governed metrics.
- **best-first search** — always expand the most promising branch (largest rate-effect) next.
- **G1–G5** — grounding rules, enforced both structurally (allow-lists, tools) and behaviorally (prompts).
- **MAX_REFUTE_ROUNDS, MIN_RATE_EFFECT, etc.** — the tunable constants in `agents/config.py` (§9).

When to open the real doc

Open `pdf/docs/architecture/AGENT_ARCHITECTURE.pdf` when you need the full `Finding` schema (§4.3), the transition table (§5.1), each agent's system-prompt sketch and logic (§6), the Critic's refutation battery (§6.5), the tunable constants (§9), or the end-to-end run trace (§10).